

Start coding or [generate](#) with AI.

```
import pandas as pd
import nltk
import re
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_s

try:
    nltk.data.find('corpora/stopwords')
except LookupError:
    nltk.download('stopwords')

print("Libraries imported successfully.")
```

```
Libraries imported successfully.
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data]   Unzipping corpora/stopwords.zip.
```

```
from google.colab import files
import os

# Define the filename
FILE_NAME = 'nlp 11.3.csv'

# Upload the file if it doesn't exist
if not os.path.exists(FILE_NAME):
    uploaded = files.upload()
    # Add a check to ensure uploaded is not None
    if uploaded is not None:
        for fn in uploaded.keys():
            print(f'User uploaded file "{fn}"')
    else:
        print("No file was uploaded or upload was cancelled.")
else:
    print(f"{FILE_NAME} already exists.")
```

```
nlp 11.3.csv already exists.
```

```
import pandas as pd

df = pd.read_csv(FILE_NAME)

print("First 5 samples of the dataset:")
```

```

print(df.head())

print("\nInput text column: 'text', Target labels column: 'label'")

print(f"\nDataset size (rows, columns): {df.shape}")

print("\nClass distribution of target labels:")
print(df['label'].value_counts())

```

```

    title \
's Fear
Pol...
ympathy
r ag...
Matters

el Unnamed: 4 \
KE      NaN
KE      NaN
AL      NaN
KE      NaN
AL      NaN

... Unnamed: 131 \
...      NaN
...      NaN
...      NaN
...      NaN
...      NaN
...      NaN

named: 136 \
      NaN
      NaN
      NaN
      NaN
      NaN

```

3161
3154

```

2
2
...
e mountain. Thus 1
1
1
1
erly man and a girl 1

(24,25,26,27,28,29,30,31,32,33,34,35,36,37,38,39,40,41,42,43,44,45,46,47,48,49

```

```

import re
from nltk.corpus import stopwords

# Get English stopwords
stop_words = set(stopwords.words('english'))

def preprocess_text(text):
    if not isinstance(text, str): # Handle non-string types, e.g., NaN
        return ""
    text = text.lower() # Convert to lowercase
    text = re.sub(r'^a-zA-Z\s]', '', text) # Remove punctuation and numbers
    words = text.split()
    # Remove stopwords
    words = [word for word in words if word not in stop_words]
    return " ".join(words)

# Apply preprocessing to the 'text' column
df['cleaned_text'] = df['text'].apply(preprocess_text)

# Display the first 5 rows with original and cleaned text
print("First 5 samples with original and cleaned text:")
print(df[['text', 'cleaned_text']].head())

```

```

First 5 samples with original and cleaned text:
      text \
0  Daniel Greenfield, a Shillman Journalism Fello...
1  Google Pinterest Digg LinkedIn Reddit Stumbleu...
2  U.S. Secretary of State John F. Kerry said Mon...
3  – Kaydee King (@KaydeeKing) November 9, 2016 T...
4  It's primary day in New York and front-runners...

      cleaned_text
0  daniel greenfield shillman journalism fellow f...
1  google pinterest digg linkedin reddit stumbleu...
2  us secretary state john f kerry said monday st...
3  kaydee king kaydeeking november lesson tonight...
4  primary day new york frontrunners hillary clin...

```

```

from sklearn.feature_extraction.text import TfidfVectorizer

```

```
# Instantiate TfidfVectorizer
tfidf_vectorizer = TfidfVectorizer(max_features=5000) # Limiting features to 5000

# Fit and transform the cleaned text data
X = tfidf_vectorizer.fit_transform(df['cleaned_text'])

# Print the shape of the TF-IDF matrix
print(f"Shape of TF-IDF matrix: {X.shape}")

# Get feature names and print the first 10 sample feature names
feature_names = tfidf_vectorizer.get_feature_names_out()
print("\nSample feature names (first 10):")
print(feature_names[:10])
```

Shape of TF-IDF matrix: (7795, 5000)

Sample feature names (first 10):
 ['abandon' 'abandoned' 'abc' 'abdullah' 'abedin' 'ability' 'able'
 'abortion' 'abortions' 'abroad']

```
from sklearn.model_selection import train_test_split

y = df['label']

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Print the shapes of the resulting datasets
print(f"Shape of X_train: {X_train.shape}")
print(f"Shape of X_test: {X_test.shape}")
print(f"Shape of y_train: {y_train.shape}")
print(f"Shape of y_test: {y_test.shape}")
```

Shape of X_train: (6236, 5000)
 Shape of X_test: (1559, 5000)
 Shape of y_train: (6236,)
 Shape of y_test: (1559,)

```
from sklearn.naive_bayes import MultinomialNB
import numpy as np

# Identify and remove NaN values from y_train and corresponding X_train
nan_indices = np.where(y_train.isna())[0]
X_train_cleaned = np.delete(X_train.toarray(), nan_indices, axis=0)
y_train_cleaned = y_train.dropna()

# Instantiate Multinomial Naive Bayes classifier
model = MultinomialNB()

# Train the model on the cleaned training data
model.fit(X_train_cleaned, y_train_cleaned)
```

```
# Print the trained model's parameters
print("Trained Multinomial Naive Bayes model parameters:")
print(model.get_params())
```

```
Trained Multinomial Naive Bayes model parameters:
{'alpha': 1.0, 'class_prior': None, 'fit_prior': True, 'force_alpha': True}
```

```
import numpy as np
```

```
# 1. Create a boolean mask to filter the original DataFrame `df`, keeping only
mask = df['label'].isin(['REAL', 'FAKE'])
df_filtered = df[mask].copy()
```

```
# 2. Update the target variable `y` by selecting the 'label' column from `df_fi
y = df_filtered['label']
```

```
# 3. Update the feature matrix `X` by transforming the `cleaned_text` column of
X = tfidf_vectorizer.transform(df_filtered['cleaned_text'])
```

```
print("DataFrame filtered, target variable `y` updated, and feature matrix `X`
```

```
DataFrame filtered, target variable `y` updated, and feature matrix `X` re-trans
```

```
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import MultinomialNB
```

```
# 4. Re-perform the train-test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random
```

```
# 5. Instantiate a new MultinomialNB model
model_retained = MultinomialNB()
```

```
# 6. Train this new model on the newly generated X_train and y_train
model_retained.fit(X_train, y_train)
```

```
# 7. Print the shapes of the new X_train, X_test, y_train, and y_test to verify
print(f"\nShape of X_train (re-split): {X_train.shape}")
print(f"Shape of X_test (re-split): {X_test.shape}")
print(f"Shape of y_train (re-split): {y_train.shape}")
print(f"Shape of y_test (re-split): {y_test.shape}")
```

```
# 8. Print the parameters of the newly trained model
print("\nParameters of the newly trained Multinomial Naive Bayes model:")
print(model_retained.get_params())
```

```
Shape of X_train (re-split): (5052, 5000)
Shape of X_test (re-split): (1263, 5000)
Shape of y_train (re-split): (5052,)
```

Shape of y_test (re-split): (1263,)

Parameters of the newly trained Multinomial Naive Bayes model:
{'alpha': 1.0, 'class_prior': None, 'fit_prior': True, 'force_alpha': True}

```
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

# 1. Make predictions on the test set
y_pred = model_retained.predict(X_test)

# 2. Calculate and print overall accuracy
accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy: {accuracy:.4f}")

# 3. Calculate and print Precision, Recall, and F1-score with 'REAL' as the pos
precision = precision_score(y_test, y_pred, average='binary', pos_label='REAL')
recall = recall_score(y_test, y_pred, average='binary', pos_label='REAL')
f1 = f1_score(y_test, y_pred, average='binary', pos_label='REAL')

print(f"Precision (REAL): {precision:.4f}")
print(f"Recall (REAL): {recall:.4f}")
print(f"F1-score (REAL): {f1:.4f}")

# 4. Generate and print the Confusion Matrix
conf_matrix = confusion_matrix(y_test, y_pred)
print("\nConfusion Matrix:")
print(conf_matrix)
```

Accuracy: 0.8804
Precision (REAL): 0.9039
Recall (REAL): 0.8578
F1-score (REAL): 0.8803

Confusion Matrix:
[[557 59]
[92 555]]

✓ Task

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
import numpy as np

# --- Step 1: Enhance Feature Extraction with N-grams ---
# Modify the TF-IDF vectorizer to include N-grams (unigrams and bigrams)
```

```
print("--- Enhancing Feature Extraction with N-grams ---")
tfidf_vectorizer_ngram = TfidfVectorizer(max_features=5000, ngram_range=(1,2))

# Fit and transform the cleaned text data from the filtered DataFrame
X_ngram = tfidf_vectorizer_ngram.fit_transform(df_filtered['cleaned_text'])

print(f"Shape of TF-IDF matrix with N-grams: {X_ngram.shape}")

# Get feature names and print the first 10 sample feature names
feature_names_ngram = tfidf_vectorizer_ngram.get_feature_names_out()
print("\nSample feature names (first 10) with N-grams:")
print(feature_names_ngram[:10])

# --- Step 2: Re-split Data for New Features ---
print("\n--- Re-splitting Data for New Features ---")
# Re-perform the train-test split using the newly generated feature matrix X_ngram
X_train_ngram, X_test_ngram, y_train_ngram, y_test_ngram = train_test_split(X_ngram, y,
                                     test_size=0.2, random_state=42)

# Print the shapes of the resulting datasets
print(f"Shape of X_train (with N-grams): {X_train_ngram.shape}")
print(f"Shape of X_test (with N-grams): {X_test_ngram.shape}")
print(f"Shape of y_train (with N-grams): {y_train_ngram.shape}")
print(f"Shape of y_test (with N-grams): {y_test_ngram.shape}")

# --- Step 3: Train Logistic Regression Model ---
print("\n--- Training Logistic Regression Model ---")
# Instantiate a Logistic Regression classifier
# Using 'liblinear' solver for binary classification with L1/L2 regularization and
# Set max_iter for convergence
log_reg_model = LogisticRegression(max_iter=1000, random_state=42, solver='liblinear')

# Train the model on the X_train_ngram and y_train_ngram data
log_reg_model.fit(X_train_ngram, y_train_ngram)

print("Trained Logistic Regression model parameters:")
print(log_reg_model.get_params())

# --- Step 4: Evaluate Logistic Regression Model ---
print("\n--- Evaluating Logistic Regression Model ---")
# Make predictions on the test set
y_pred_log_reg = log_reg_model.predict(X_test_ngram)

# Calculate and print overall accuracy
```

```

accuracy_log_reg = accuracy_score(y_test_ngram, y_pred_log_reg)
print(f"Accuracy (Logistic Regression): {accuracy_log_reg:.4f}")

# Calculate and print Precision, Recall, and F1-score with 'REAL' as the positive class
precision_log_reg = precision_score(y_test_ngram, y_pred_log_reg, average='binary')
recall_log_reg = recall_score(y_test_ngram, y_pred_log_reg, average='binary', pos_label='REAL')
f1_log_reg = f1_score(y_test_ngram, y_pred_log_reg, average='binary', pos_label='REAL')

print(f"Precision (REAL, Logistic Regression): {precision_log_reg:.4f}")
print(f"Recall (REAL, Logistic Regression): {recall_log_reg:.4f}")
print(f"F1-score (REAL, Logistic Regression): {f1_log_reg:.4f}")

# Generate and print the Confusion Matrix
conf_matrix_log_reg = confusion_matrix(y_test_ngram, y_pred_log_reg)
print("\nConfusion Matrix (Logistic Regression):")
print(conf_matrix_log_reg)

# --- Step 5: Analyze Performance and Plan Next Steps ---
print("\n--- Performance Analysis and Next Steps ---")
target_accuracy = 0.95
if accuracy_log_reg > target_accuracy:
    print(f"Target accuracy of >{target_accuracy} was achieved with Logistic Regression.")
else:
    print(f"Target accuracy of >{target_accuracy} was NOT achieved with Logistic Regression.")
    print("\nPotential next steps to improve the model performance:")
    print("1. Hyperparameter Tuning: Optimize hyperparameters of the Logistic Regression model.")
    print("2. Explore other models: Consider more advanced models like Support Vector Machines or Random Forests.")
    print("3. Feature Engineering: Experiment with different TF-IDF parameters (e.g., ngram_range).")
    print("4. Data Augmentation: If more data is available, increasing the training dataset size.")

# --- Step 6: Final Task Summary ---
print("\n--- Final Task Summary ---")
print("Text Classification Process Summary:")
print("Dataset Characteristics: The dataset contains news articles with 'text' column for content and 'category' for labels.")
print("Preprocessing Steps: Text was cleaned by converting to lowercase, removing stopwords, and tokenizing.")
print("Feature Extraction: TF-IDF with unigrams and bigrams (ngram_range=(1,2)) was used to generate features.")
print("Model Used: A Logistic Regression classifier was trained on the preprocessed data.")
print("Key Findings from Evaluation:")
print(f"- The Logistic Regression model achieved an accuracy of {accuracy_log_reg:.4f}.")
print(f"- Precision for 'REAL' class: {precision_log_reg:.4f}")
print(f"- Recall for 'REAL' class: {recall_log_reg:.4f}")
print(f"- F1-score for 'REAL' class: {f1_log_reg:.4f}")

```

```
print(f"- The model's performance ({accuracy_log_reg:.4f}) did {'not ' if accuracy_
print("Further steps to improve the model performance include hyperparameter tuning
```

```
from sklearn.feature_extraction.text import TfidfVectorizer

# 1. Instantiate TfidfVectorizer with ngram_range=(1,2)
tfidf_vectorizer_ngram = TfidfVectorizer(max_features=5000, ngram_range=(1,2))

# 2. Fit and transform the cleaned text data from df_filtered
X_ngram = tfidf_vectorizer_ngram.fit_transform(df_filtered['cleaned_text'])

# 3. Print the shape of the TF-IDF matrix with n-grams
print(f"Shape of TF-IDF matrix with n-grams: {X_ngram.shape}")

# 4. Get feature names
feature_names_ngram = tfidf_vectorizer_ngram.get_feature_names_out()

# 5. Print the first 10 sample feature names
print("\nSample feature names with n-grams (first 10):")
print(feature_names_ngram[:10])
```

Shape of TF-IDF matrix with n-grams: (6315, 5000)

Sample feature names with n-grams (first 10):
 ['abandon' 'abandoned' 'abc' 'abedin' 'ability' 'able' 'abortion'
 'abortions' 'abroad' 'absence']

```
from sklearn.model_selection import train_test_split

# Re-perform the train-test split using the newly generated feature matrix X_ngram
X_train_ngram, X_test_ngram, y_train_ngram, y_test_ngram = train_test_split(X_ngram, y,
                                     test_size=0.2, random_state=42)

# Print the shapes of the resulting datasets
print(f"Shape of X_train (with N-grams): {X_train_ngram.shape}")
print(f"Shape of X_test (with N-grams): {X_test_ngram.shape}")
print(f"Shape of y_train (with N-grams): {y_train_ngram.shape}")
print(f"Shape of y_test (with N-grams): {y_test_ngram.shape}")
```

Shape of X_train (with N-grams): (5052, 5000)
 Shape of X_test (with N-grams): (1263, 5000)
 Shape of y_train (with N-grams): (5052,)
 Shape of y_test (with N-grams): (1263,)

```
from sklearn.linear_model import LogisticRegression

# Instantiate a Logistic Regression classifier
# Using 'liblinear' solver for binary classification with L1/L2 regularization
# Set max_iter for convergence
log_reg_model = LogisticRegression(max_iter=1000, random_state=42, solver='liblinear')
```

```
# Train the model on the X_train_ngram and y_train_ngram data
log_reg_model.fit(X_train_ngram, y_train_ngram)

print("Trained Logistic Regression model parameters:")
print(log_reg_model.get_params())
```

```
Trained Logistic Regression model parameters:
{'C': 1.0, 'class_weight': None, 'dual': False, 'fit_intercept': True, 'intercep
```

```
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_s

# Make predictions on the test set
y_pred_log_reg = log_reg_model.predict(X_test_ngram)

# Calculate and print overall accuracy
accuracy_log_reg = accuracy_score(y_test_ngram, y_pred_log_reg)
print(f"Accuracy (Logistic Regression): {accuracy_log_reg:.4f}")

# Calculate and print Precision, Recall, and F1-score with 'REAL' as the positi
precision_log_reg = precision_score(y_test_ngram, y_pred_log_reg, average='bina
recall_log_reg = recall_score(y_test_ngram, y_pred_log_reg, average='binary', p
f1_log_reg = f1_score(y_test_ngram, y_pred_log_reg, average='binary', pos_label

print(f"Precision (REAL, Logistic Regression): {precision_log_reg:.4f}")
print(f"Recall (REAL, Logistic Regression): {recall_log_reg:.4f}")
print(f"F1-score (REAL, Logistic Regression): {f1_log_reg:.4f}")

# Generate and print the Confusion Matrix
conf_matrix_log_reg = confusion_matrix(y_test_ngram, y_pred_log_reg)
print("\nConfusion Matrix (Logistic Regression):")
print(conf_matrix_log_reg)
```

```
Accuracy (Logistic Regression): 0.9327
Precision (REAL, Logistic Regression): 0.9460
Recall (REAL, Logistic Regression): 0.9212
F1-score (REAL, Logistic Regression): 0.9334
```

```
Confusion Matrix (Logistic Regression):
[[582  34]
 [ 51 596]]
```

```
from sklearn.model_selection import GridSearchCV
from sklearn.linear_model import LogisticRegression

# 1. Define a parameter grid
param_grid = {
    'C': [0.01, 0.1, 1, 10, 100],
    'penalty': ['l1', 'l2']
}
```

```
# 2. Instantiate a Logistic Regression model
log_reg = LogisticRegression(solver='liblinear')

# 3. Instantiate GridSearchCV
# Using 'f1_macro' for scoring as it considers both precision and recall
grid_search = GridSearchCV(estimator=log_reg,
                             param_grid=param_grid,
                             scoring='f1_macro')

# 4. Fit the GridSearchCV object to your training data
print("\nStarting GridSearchCV...")
grid_search.fit(X_train_ngram, y_train_ngram)
print("GridSearchCV completed.")

# 5. Print the best parameters and best cross-validation score
print("\nBest parameters found by GridSearchCV:")
print(grid_search.best_params_)

print("\nBest cross-validation score (f1_macro):")
print(f"{grid_search.best_score_:.4f}")
```

```
Starting GridSearchCV...
Fitting 5 folds for each of 10 candidates, totalling 50 fits
GridSearchCV completed.
```

```
Best parameters found by GridSearchCV:
{'C': 10, 'penalty': 'l2'}
```

```
Best cross-validation score (f1_macro) during grid search:
0.9261
```

```
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

# 6. Get the best estimator from GridSearchCV
best_log_reg_model = grid_search.best_estimator_

# 7. Make predictions on the test set using the best estimator
y_pred_tuned = best_log_reg_model.predict(X_test_ngram)

# 8. Calculate and print overall accuracy
accuracy_tuned = accuracy_score(y_test_ngram, y_pred_tuned)
print(f"\nAccuracy (Tuned Logistic Regressor): {accuracy_tuned:.4f}")

# 9. Calculate and print Precision, Recall, and F1-score
precision_tuned = precision_score(y_test_ngram, y_pred_tuned)
recall_tuned = recall_score(y_test_ngram, y_pred_tuned)
f1_tuned = f1_score(y_test_ngram, y_pred_tuned)

print(f"Precision (REAL, Tuned Logistic Regressor): {precision_tuned:.4f}")
print(f"Recall (REAL, Tuned Logistic Regressor): {recall_tuned:.4f}")
print(f"F1-score (REAL, Tuned Logistic Regressor): {f1_tuned:.4f}")

# 10. Generate and print the Confusion Matrix
```

```
conf_matrix_tuned = confusion_matrix(y_test_r  
print("\nConfusion Matrix (Tuned Logistic Reg  
print(conf_matrix_tuned)
```

```
Accuracy (Tuned Logistic Regression): 0.9438  
Precision (REAL, Tuned Logistic Regression): 0.9528  
Recall (REAL, Tuned Logistic Regression): 0.9366  
F1-score (REAL, Tuned Logistic Regression): 0.9447
```

```
Confusion Matrix (Tuned Logistic Regression):  
[[586  30]  
 [ 41 606]]
```